

OPERATING SYSTEMS — MODULE 2: PROCESS MANAGEMENT & SYNCHRONIZATION

1. What is a Process?

A process is a program in execution. While a program is a passive entity stored on disk (executable file), a process is an active entity with a program counter specifying the next instruction to execute and a set of associated resources.

Process Memory Layout:

- **Text Section:** Contains the compiled program code.
- **Data Section:** Stores global and static variables.
- **Heap:** Memory dynamically allocated during process runtime (e.g., malloc in C or new in Java).
- **Stack:** Temporary data storage for function parameters, return addresses, and local variables.

2. Process States & Life Cycle

As a process executes, it changes state:

1. **New:** The process is being created.
2. **Ready:** The process is waiting to be assigned to a processor.
3. **Running:** Instructions are being executed on the CPU.
4. **Waiting (Blocked):** The process is waiting for some event to occur (such as I/O completion or reception of a signal).
5. **Terminated:** The process has finished execution.

3. Process Control Block (PCB)

Each process is represented in the operating system by a Process Control Block (also called a Task Control Block):

- **Process ID (PID):** Unique integer identifier.
- **Process State:** Current state (New, Ready, Running, etc.).
- **Program Counter (PC):** Address of the next instruction to execute.
- **CPU Registers:** Accumulators, index registers, stack pointers.
- **CPU-Scheduling Information:** Priority, scheduling queue pointers.
- **Memory-Management Information:** Base/limit registers, page tables.
- **Accounting Information:** CPU time used, time limits.
- **I/O Status Information:** List of open files and allocated I/O devices.

4. CPU Scheduling Algorithms

The goal of CPU scheduling is to maximize CPU utilization, throughput, and minimize turnaround time, waiting time, and response time.

1. **First-Come, First-Served (FCFS):**

- Non-preemptive. The process that requests the CPU first gets it first.
- Disadvantage: Convoy Effect (short processes stuck waiting behind a massive I/O or CPU-bound process).

2. **Shortest Job First (SJF):**

- Associates each process with the length of its next CPU burst.
- SJF is mathematically optimal (gives minimum average waiting time).
- Problem: Cannot know the exact length of the future CPU burst in advance; prone to starvation for long processes.

3. **Round Robin (RR):**

- Preemptive scheduling designed especially for time-sharing systems.
- Defines a small unit of time called a **Time Quantum** (typically 10 to 100 milliseconds).
- If the time quantum is too large, RR behaves like FCFS. If too small, context-switch overhead becomes excessive.

5. Critical Section Problem & Semaphores

A Critical Section is a segment of code where a process accesses shared resources (like shared memory, global variables, or files).

Three Mandatory Requirements for any valid solution:

1. **Mutual Exclusion:** If process P is executing in its critical section, no other processes can execute in their critical sections.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter, only those processes not in their remainder sections can participate in deciding who enters next.
3. **Bounded Waiting:** A bound must exist on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter.

Semaphores:

A synchronization tool introduced by Edsger Dijkstra. An integer variable accessed only through two standard atomic operations:

- `wait()` (or `P()` operation): decrements semaphore value. If negative, the process blocks.
- `signal()` (or `V()` operation): increments semaphore value. If there are waiting processes, one is unblocked.

6. Deadlocks

A deadlock occurs when a set of blocked processes each holds a resource and waits to acquire a resource held by another process in the set.

The 4 Necessary Coffman Conditions for Deadlock:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode.

2. ****Hold and Wait:**** A process must be holding at least one resource and waiting to acquire additional resources held by other processes.
3. ****No Preemption:**** Resources cannot be preempted; a resource can be released only voluntarily by the process holding it.
4. ****Circular Wait:**** A closed chain of processes exists such that each process holds at least one resource needed by the next process in the chain.